

VIT CHENNAI · DEPT. OF ECE, SENSE

Runtime Parameter Adaptation for fq_codel

A measured evaluation on stock Linux — and what the measurements had to survive

Amritha S · Yugeshwaran P · Deepti Annuncia

The question

fq_codel is the default queue discipline on most Linux systems. Four of its parameters are fixed when you configure it and never change afterwards.

target

5 ms

acceptable sojourn time

interval

100 ms

CoDel observation window

limit

10240 p

maximum queue depth

quantum

1514 B

per-flow service quantum

Does adapting them at runtime beat leaving them alone?

We built the controller, built the apparatus to test it honestly, and measured. The answer matters either way.

DESIGN RATIONALE

Why we chose what we chose

Every choice below was forced by something that went wrong, or by something that could go wrong undetected.

Why a three-node topology, not two

CHOICE · client → router → server, with the AQM on the router's egress

The two-node setup

Bottleneck on veth1 inside ns1; iperf3 runs from ns2.
Data flows ns2 → ns1, but an egress qdisc on veth1 shapes
ns1 → ns2.

So it shapes the ACK path, not the data path.

Measured: 9,777 Mbps unshaped

66 bytes/packet — pure TCP ACKs

~130,000 drops/s on an 826 pkt/s link

The three-node router

TBF + AQM on the router's egress toward the server.
Data traverses it, so the bottleneck is on the data path.

Measured: 9.38 Mbps shaped

1509 bytes/packet — real data

~19 drops/s — physically plausible

Diagnostic worth keeping: bytes ÷ packets on the monitored queue. 66 means you are queueing acknowledgements.

Why we measure latency two different ways

CHOICE · a 20 Hz sparse probe AND in-band bulk-flow RTT from TCP_INFO

fq_codel's new-flow heuristic deliberately privileges sparse flows. A ping is a sparse flow. So a ping measures the best case, not what the bulk traffic sees.

Sparse ICMP probe

22.4 ms

what an interactive flow sees

Bulk TCP flows

36.3 ms

what the data flows actually see

A 62% difference. Quoting only the probe — as the original work did — overstates the benefit to bulk traffic. Both belong in a latency claim.

Why we run a controller that does nothing

CHOICE · a sham condition — the controller polls at the same cadence but applies no change

The controller is an extra process polling every 500 ms. In a 2-vCPU emulated VM that alone can add latency. Without a control condition, "the controller made it slower" and "the controller's decisions made it slower" are indistinguishable.

1 static fq_codel

no controller at all

baseline

2 + sham controller

polls, changes nothing

isolates CPU cost

3 + ACAPE

polls and adapts

isolates the decisions

No published adaptive-AQM paper we found runs this control. It is the reason our null result is credible rather than merely unexplained.

Why we read BPF maps with `bpf(2)`, not `bpftool`

CHOICE · in-process syscalls instead of spawning a helper each tick

bpftool per tick

~2.5 s per call under a 9p root
stretched the 0.5 s control interval to 3.17 s
a 20 s run yielded only 7 samples

bpf(2) in process

tick restored to 0.709 s — 4.5× faster
no extra process, so no CPU confound
raw bytes, so no hex-string decode bug

The second reason matters more than the first.

Spawning a helper process to measure a system whose latency you are measuring is the same confound the sham condition exists to remove. Reading the map in process avoids introducing it at all.

Why the workload changes during the run

CHOICE · a staged load — 2 flows, then 24, then back to 2

A controller that classifies congestion into four states can only be tested by a workload that visits more than one of them.

Constant overload (original)

HEAVY in 80.9% of samples

MODERATE in 0.04%, LIGHT never

A four-state classifier using two states

Staged load (ours)

Flow count changes twice during the run

The regime actually transitions

The adaptive logic is exercised, not just saturated

Under constant overload the original controller ratcheted target to its floor every run and stopped — always exactly 15 adjustments, which is just the number of $\times 0.9$ steps from 5 ms to 1 ms. That is a constant, not a measurement.

Why these metrics and these baselines

1

Goodput from `sum_received`

iperf3's `sum_sent` counts bytes handed to the socket. Under pfifo it read 12.16 Mbps on a 10 Mbit link — the sender filling a bloated buffer. Only `sum_received` crossed the bottleneck.

2

Nine queue disciplines

pfifo, SFQ, RED, CoDel, PIE, FQ-PIE, CAKE, fq_codel, fq_codel+ACAP. A comparison that omits the obvious alternatives is not a comparison — and PIE and CAKE are genuinely competitive.

3

Three repetitions, exact statistics

Confidence intervals from Student's *t* via the incomplete beta function, not a normal approximation — which at $n=3$ overstates significance and flipped one of our own conclusions.

Each of these was added because its absence had already produced a wrong number.

RESULTS

What the measurements show

Stock Linux · 10 Mbit bottleneck · 8 TCP CUBIC flows · 20 ms base RTT · 3 repetitions

The effect that dominates everything

Presence or absence of a flow-queueing AQM — not the tuning of one

2371 ms

p95 RTT
pfifo, no AQM

25 ms

p95 RTT
fq_codel

~100×

latency reduction

0%

goodput cost

Two orders of magnitude, at no throughput or fairness cost.

Every flow-queueing discipline with a delay target — fq_codel, FQ-PIE, CAKE — lands near 23 ms. Single-queue AQMs (CoDel, PIE) land near 45 ms. SFQ, which has fair queueing but no AQM, bloats to 115 packets. The AQM and the scheduler each do a distinct job, and both matter far more than parameter choice.

Does adapting the parameters help?

On the evidence so far: it shrinks the queue slightly, and does not reduce the latency users see.

Mean backlog	−15%	not distinguishable ($p = 0.066$)
p95 queue delay	−16%	marginal ($p = 0.047$)
p95 RTT	+1%	not distinguishable ($p = 0.449$)
Mean RTT	+2.7%	significant — and in the wrong direction
Goodput / fairness	0%	no cost

The queue under static fq_codel was already only ~7 packets. There was very little delay left to remove, and tightening target costs more drop-and-retransmit than it saves.

INTEGRITY

What did not survive verification

Seven classes of silent failure. Each produced results that were confident, internally consistent, and wrong.

The seven defects

1

Loopback traffic

Parts 1-3 ran iperf3 to 127.0.0.1 at 93-219 Gbit/s. No qdisc was ever in the path.

2

ACK-path bottleneck

Parts 2-4 shaped the acknowledgement direction. 66 bytes/packet.

3

No RTT ever measured

The reported 0.541 / 2.200 / 2.445 ms have no source anywhere in the repository.

4

eBPF telemetry read zero

Three decode bugs: hex strings into a bare except, wrong struct offsets, CLOCK_REALTIME vs CLOCK_MONOTONIC (age = 56.8 years).

5

Elephant threshold unreachable

10 MB threshold, 9.38 MB maximum possible flow share. The classifier could never fire.

6

Prediction had no effect

All 375 logged adjustments applied the identical action.

7

Headline figures hardcoded

"12× faster" is the literal array [120, 70, 60, 5]. No stabilisation time was ever measured.

All seven are pinned by regression tests that fail against the original behaviour.

How each was caught

Not one was visible from a summary statistic. Every one was found by checking a physical invariant.

Bytes ÷ packets on the monitored queue

66 B means you are queueing ACKs, not data

Drop rate vs the link's packet rate

A 10 Mbit link passes 826 pkt/s; 130,000 drops/s is impossible

Does varying a control input change the output?

If not, the control path is inert however it is labelled

Is the threshold reachable at all?

10 MB elephant threshold, 9.38 MB maximum flow share

Does the reported value reach the kernel?

A float in userspace is not the qdisc parameter

We think this is the most transferable part of the work.

Where this sits against recent work

2023–2026 · what is taken, and what is not

Already done — cannot claim

- DESiRED (2024) adapts an AQM target at runtime with DRL + INT on P4
- Adaptive RED (2001) is the AIMD rule we borrow
- ACoDel (2020) adapts CoDel's interval — with a stability proof we lack
- QueuePilot (2023), AQM-LLM (2026) learn AQM policies

Not done by anyone — our claim

- A sham-controller condition separating CPU cost from decisions
- Sparse-probe and bulk-flow latency reported separately
- Comparison against eight alternative disciplines
- A published null result for the adaptation itself

The claim we can defend

The first controlled evaluation of runtime parameter adaptation for a flow-queueing AQM on stock Linux — and the finding that it does not measurably help.

CONCLUSION

What we are left with

1

Flow queueing with a delay target is what matters

~100× tail-latency reduction at zero goodput cost

2

Tuning its parameters is not

No distinguishable improvement over the defaults in the regimes tested

3

The methodology is the contribution

Seven silent failure classes, the invariants that catch them, and a released artefact